

入门数据结构

本文同步发表于 [QOJ](#)。

教学计划

- 分块
- 线段树
- 树状数组
- ST 表

引入：区间求和问题

多次查询一个长度为 10^5 的序列的某个子段 $[l_i, r_i]$ 的和。我们很容易写下如下代码：

```
for(int i=0;i<q;++i) {  
    int sum=0;  
    for(int j=ql[i];j<=qr[i];++j)  
        sum+=a[j];  
    cout<<sum<<"\n";  
}
```

一个子段可以拆分为两个从序列开头开始的子段的差，所以我们只需要记录每个 $\sum_{i=1}^r a_i$ ，相减得到答案。前缀和可以解决一系列具有结合律和可减性的问题。

换句话说，只要满足 $f(l, r) = f(l, k) + f(k + 1, r)$ 和 $f(l, r) = f(1, r) - f(1, l - 1)$ ，其中加号和减号可以代表任意的一组逆运算，而不一定是加法和减法。

单点修改：分块

现在将原问题改为如下问题：

- 多次查询一个长度为 10^5 的序列的某个子段 $[l_i, r_i]$ 的和。
- 随时修改某一个 a_i 。

怎么做？如果记录所有的前缀和，那么每次修改最多可能要修改 n 个前缀和！我们需要记录那么多前缀和吗？

每隔 B 个数划分一段，只记录每段内的前缀和，查询的复杂度是多少？
(考虑中间 $O(n/B)$ 个完整块和两边的散点)

$$O(n/B)$$

每次修改需要最多修改 B 个。所以复杂度？

$$O(n/B + B)$$

对勾函数。取 $B = \sqrt{n}$ 平衡复杂度。

整体修改

现在将原问题改为如下问题：

- 多次查询一个长度为 10^5 的序列的某个子段 $[l_i, r_i]$ 的和。
- 随时修改某一个 a_i 。
- 将所有的数增加一个 k_i 。

看上去似乎不是很可做，但是仔细想一下，我们真的需要维护第三种操作吗？

事实上，我们只需要知道第三个操作的全体对于整体的影响，也就是所有的数被第三个操作加上了多少。记录一个标记 T_i ，每次查询的时候给答案增加 $(r - l + 1) \cdot T_i$ 就行了。

让我们修改区间！

现在继续加强原问题：

- 多次查询一个长度为 10^5 的序列的某个子段 $[l_i, r_i]$ 的和。
- 随时给某一段区间 $[l_i, r_i]$ 加上 c_i 。

我们需要记录某一个块的所有前缀和吗？如果只记录整个块的和呢？

$$O(n/B + B) = O(\sqrt{n})$$

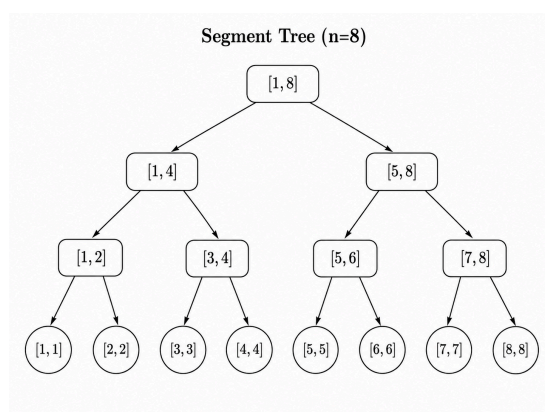
我们考虑修改仍是中间 $O(n/B)$ 个完整块和两边的 $O(B)$ 个散点。散点处理是容易的，考虑对每个整块我们仍然需要在 $O(1)$ 的时间内解决？

记录一个**标记** T_i 表示比起记录的散点值，这个区间作为一个整体的实际值比它要多多少。

这样就可以愉快地区间修改了！

线段树：分治的永久化

我们考虑使用一棵 $k + 1$ 层满二叉树维护信息。



我们都在学习堆的时候学过满二叉树的二进制编号存储方案：

- 根节点的编号为 1。
- 节点 x 左右儿子的编号分别为 $2x$ 和 $2x + 1$ 。

其中，我们认为：

- 编号第 c 小的叶子 ($2^k + c - 1$ 号点) 代表一个长度为 1 的区间 $[c, c]$ ，
- 对于一个非叶子节点 u ，它代表的区间为 $2u$ 和 $2u + 1$ 代表的区间的并。

我们可以注意到，这个并区间就是以 u 为根的子树的所有叶子代表的单点构成的并集。

接下来我们证明如下 Lemma：

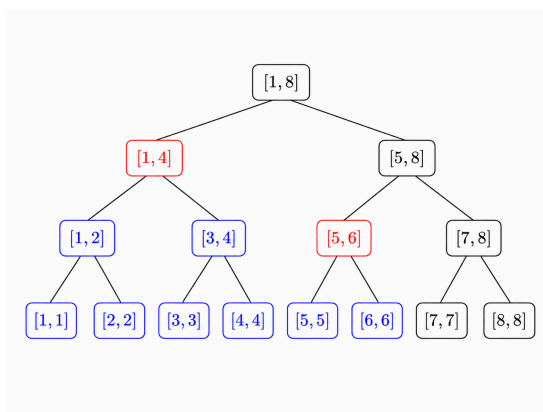
- 在这个二叉树结构上，一个区间最少可以被拆分为至多不超过 $2k + 2$ 个点所代表的区间的并集。

考虑如下一个事实：

- 我们选出的一定是所有包含在查询区间内的极大的区间，这样的选取是唯一的。

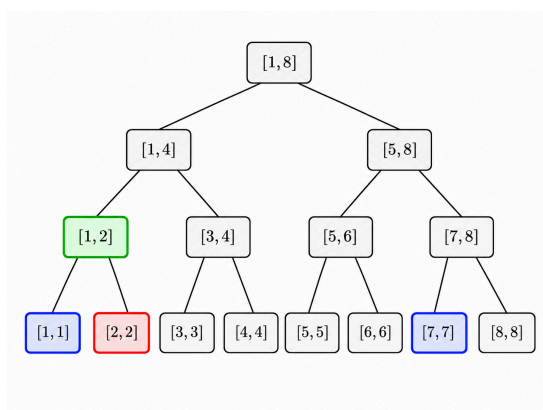
我们证明这是对的：

- 唯一性：一个节点代表的区间被选出当且仅当它的父亲不完全被查询区间包含，并且它自己被包含。这样的选取显然是唯一的。
- 合法性：我们实际上只需要证明选出的区间不交。显然如果相交，一定是选了一个点是另一个点的祖先，与前面的要求矛盾。
- 最优性：任取一个其他的拆分方式，若有一个该方案中的节点不被选取，则首先不可能取它的祖先，其次必须用它的真子树节点拼出这个区间，一定不优于直接选取。



我们考察这样一种选取方案在第 l 层的选取节点个数：

- 假设某种选取方案选取了不低于 3 个同层节点，则必然存在两个相对父亲的 **identity** 相同（即这个节点是左/右儿子）的节点，他们中间必然夹着 $2k + 1$ 个同层节点，我们可以把这个问题归约到一个长度为奇数的区间在叶子层的选取问题，由于不是叶子的节点代表的区间长度必然为**偶数**，因此必然会选取一个**叶子**。而我们考察一个奇数长度的区间的所有叶子必然存在一个最左侧的右儿子或最右侧的左儿子，它的所有祖先都不可选择。因此必然选择一个**边界上的叶子**，且这个边界叶子的**兄弟**在原方案中也被选择，与最优性矛盾。因此我们证明了在最优方案中，每一层最多选 2 个节点。



那么如何寻找这样的区间呢？我们考虑自顶而下递归，如果当前的节点被访问到了，且它完全被包含在内，那么选它，且它的子树内的其他区间都不会被选择；否则递归下去找子树。

如下算法实现了这个功能：

```
int query(int ql,int qr,int l,int r,int u) {
    if(ql<=l && r<=qr) {
        // 该节点完全包含于查询区间
        return sum[u];
    }
    // 当前节点不完全包含于查询区间，需要继续递归下去找子树中的节点
    int mid=l+(r-l)/2,ans=0;
    if(ql<=mid) ans+=query(ql,qr,l,mid,u*2); // 和查询区间无交，不继续递归
    if(qr>mid)  ans+=query(ql,qr,mid+1,r,u*2+1);
    return ans;
}
```

我们考察这个算法第一次同时向两侧子树递归的点 r ：

- 对于左子树，查询区间变为一个后缀，因此所有向右递归的情况都会直接返回；
- 对于右子树，查询区间变为一个前缀，因此所有向左递归的情况都会直接返回。

因此我们又一次说明了在最优方案中，每一层最多选 2 个节点。

我们只说明了选取的点是 $O(\log n)$ 的，但是我们没有说明它们的祖先也是 $O(\log n)$ 个的，怎么办呢？考虑这个算法的执行过程：

- 它在 r 之后的递归次数，我们已经证明是 $O(\log n)$ 的；
- 它在 r 之前只会访问到 r 的祖先，显然只有 $O(\log n)$ 个。

因此我们证明了这个算法可以用于实现一个 $O(k) = O(\log n)$ 的区间查询。又因为这个算法只会走祖先，所以它们的祖先个数也是 $O(\log n)$ 的。

对于单点修改，我们发现，被影响到的区间个数只有 $O(k) = O(\log n)$ 个，因此直接一条链修改上去即可。

那么区间修改怎么做呢？

我们还是考虑对所有的区间打标记。但对树上进行操作，要注意两件事情：

- 对子树的影响；
- 对祖先的影响。

对于前者，我们发现，对一个深度为 d 的节点所代表的区间来说，它的 d 个祖先的标记都会影响到这个点的实际值。

并且它们是可以加起来算的！所以我们只要递归的过程中记录这些标记的和就可以了！

对于后者似乎不太好弄。因为查询子树内的标记是一件不可能的事情，所以我们考虑对于一个要打标记的修改，对修改区间的那些选取的点的祖先存储的值进行**实际修改**，以避免子树标记的影响。

于是我们就在 $O(\log n)$ 的复杂度内实现了区间修改区间查询。

```

void update(int ql,int qr,int qadd,int l,int r,int u) {
    sum[u]+=(qr-ql+1)*qadd;
    if(ql<=l && r<=qr) { tag[u]+=qadd; return; }
    int mid=(l+r)>>1;
    if(ql<=mid) update(ql,min(mid,qr),qadd,l,mid,u*2);
    if(qr>mid) update(max(mid+1,ql),qr,qadd,mid+1,r,u*2+1);
}
int query(int ql,int qr,int tagsum,int l,int r,int u) {
    if(ql<=l && r<=qr) return sum[u]+(qr-ql+1)*tagsum;
    int mid=(l+r)>>1,ans=0; tagsum+=tag[u];
    if(ql<=mid) ans+=update(ql,min(mid,qr),tagsum,l,mid,u*2);
    if(qr>mid) ans+=update(max(mid+1,ql),qr,tagsum,mid+1,r,u*2+1);
    return ans;
}

```

标记下传

我们刚才进行标记永久化，利用的是标记代表的信息可以按任意顺序累加这一点。如果不满足这一条件呢？

我们来看这样一道题：

- 区间加；
- 区间查询 $\sum_{i=l}^r f(a_i)$ ，其中 $f(n) = \frac{1}{\sqrt{5}} \cdot \left(\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right)$ 为第 n 个斐波那契数列的项。

我们先考虑单点加该怎么做。根据线性代数的知识，我们可以利用矩阵 $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$ 及其预处理后的幂来进行对斐波那契和单点加的维护。

对于区间修改，我们尝试标记永久化。我们发现，此时标记永久化的思路不可行，因为标记代表的信息不能交换处理顺序。

同学们可以拿长度为 2 的区间试一下。

我们可能会想，如果我们保证访问到一个点的 `val` 的时候，保证这个点没有标记的影响，那该多好啊！这个时候就要请出我们的下一个 trick 了：标记下传。

具体而言，我们发现，只要我们正确维护了标记对于当前点的 `val` 的影响，我们可以通过继续修改儿子的 `val`，使得自己的 `tag` 变为两个等效的儿子的 `tag`。如果儿子本身有 `tag`，这个 `tag` 需要可以和之前的 `tag` 合并。

那么我们只需要保证在每次访问到这个点的 `val` 之前，都保证这个点已经被 `pushdown` 过了，就可以了！

```

void pushup(int u) { st[u]=st[u*2]+st[u*2+1]; }
void pushdown(int u,int l,int r) {
    int mid=l+(r-l)/2,ln=mid-l+1,rn=r-mid;
    tag[u*2]+=tag[u]; st[u*2]+=ln*tag[u];
    tag[u*2+1]+=tag[u]; st[u*2+1]+=rn*tag[u];
    tag[u]=0;
}
void update(int ql,int qr,int qv,int l,int r,int u) {

```

```

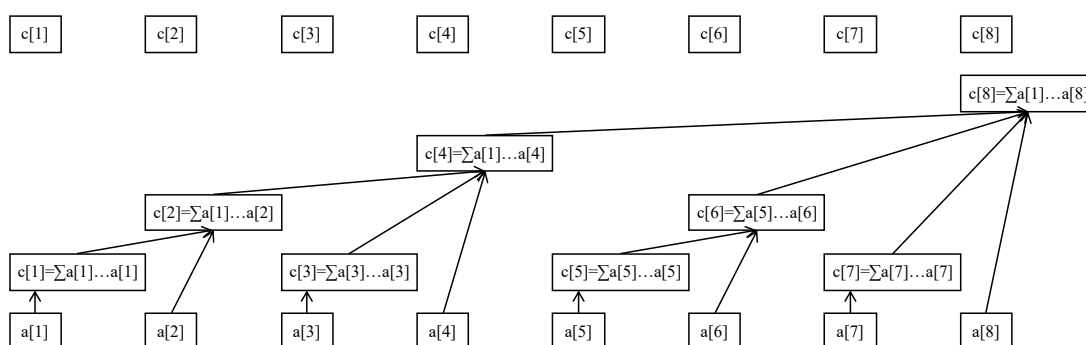
tag[u]+=qv;
if(q1<=l && r<=qr) return void(st[u]+=(r-l+1)*qv);
int mid=l+(r-l)/2; pushdown(u,l,r);
if(q1<=mid) update(q1,qr,qv,l,mid,u*2);
if(qr>mid) update(q1,qr,qv,mid+1,r,u*2+1);
return pushup(u);
}
int query(int q1,int qr,int l,int r,int u) {
if(q1<=l && r<=qr) return st[u];
int mid=l+(r-l)/2,ans=0; pushdown(u,l,r);
if(q1<=mid) ans+=query(q1,qr,l,mid,u*2);
if(qr>mid) ans+=query(q1,qr,mid+1,r,u*2+1);
return ans;
}

```

树状数组

对于那些满足区间可减性的单点加区间修改问题，我们发现本质上似乎只需要知道父亲和左儿子的答案，就可以知道右儿子的值。

所以我们可以考虑把所有的右儿子砍掉，只留左儿子，最后形成这样一个东西：



也就是说，除了根对应 $n = 2^k$ 以外，对于原树上的一个编号为偶数的点 u ，我们定义它的新编号为它所代表的区间的右端点。

我们考察这样一个事实：对于处理前的线段树，一个代表了 $[c, c]$ 的叶子 $2^k + c - 1$ ，它从根走过来的左/右的路径信息，实际上被编码在了 $c - 1$ 的二进制表示中。比如 $6 = (110)_2$ ，就代表了右-右-左，也就是图中的 $a[7]$ 。我们定义这样重新编号后的节点 x 所管辖的区间为 R_x 。我们发现， $|R_x| = \text{lowbit}(x) = 2^{\text{ffs}(x)}$ ，其中 $\text{ffs}(x)$ 表示 x 二进制表示中最低的位数。例如， $\text{ffs}(1) = 0$, $\text{ffs}(4) = 2$ 。

我们如何证明这是一个留下来的点到 $[1, n]$ 的双射呢？首先根据线段树的性质，一个点 u 表示的区间一定能写成 $(q \cdot 2^t, (q + 1) \cdot 2^t]$ 。这个节点是父亲的左儿子，当且仅当它是某个长度为 2^{t+1} 的大区间的左半边，也就是 q 是偶数。

于是对于一个被保留的非根节点，它一定是左儿子。

于是右端点满足 $r = (2k + 1) \cdot 2^t$ ，即 $\text{lowbit}(r) = r - l + 1$ 。我们就证明了映射是单射。

接下来我们证明映射是满射，对于 $1 \leq x < n$ ，令 $p = \text{lowbit}(x)$ ，考虑区间 $[x - p + 1, x]$ ，我们有 $x = (2k + 1)p$ ，也就是 $x - p + 1 = 2kp + 1$ ，因此区间可以写成 $[2kp + 1, (2k + 1)p]$ ，在线段树中是一个左儿子，会对应一个点。证毕。

那么我们进一步观察这个结构，可以发现如下性质：

- $\forall x < y, R_x \cap R_y \in \{\emptyset, R_x\}$
- $R_x \subseteq R_{x+\text{lowbit}(x)}$
- $\forall x < y < \text{lowbit}(x), R_x \cap R_y = \emptyset$

如何证明呢？第一条根据线段树节点的性质是显然的；第二条第三条考虑编号双射的定义也比较简单。我们只需要证明在新的树上， u 的父亲是 $u + \text{lowbit}(u)$ 即可。考虑如下证明：

我们还是设 $p = \text{lowbit}(u)$ ，我们已经知道 $R_u = [u - p + 1, u]$ 。由于 u 是左儿子，因此在原线段树的父亲区间是 $[u - p + 1, u + p]$ 。现在分类讨论：

- 若原线段树的父亲区间是左儿子或根，那么停止递归，返回答案，因此新的编号就是 $u + p$ ；
- 否则，继续找它的祖先。由于右儿子的右端点和父亲相同，这一步不会改变右端点 $u + p$ 。因此我们证明了 $u + p$ 就是 u 的父亲。

所以我们可以写出在这棵树上进行单点修改的代码如下：

```
for(;p<=n;p+=p&-p) c[p]+=v;
```

查询一个前缀的时候，我们注意到当前节点对应的区间是一个右端点为查询区间的右端点的区间，因此我们可以不断向前跳，进行递归。

由于当前点 u 管辖的区间为 $[u - p + 1, u]$ ，下一步区间的右端点为 $u - p$ ，所以我们可以写出如下代码：

```
for(;p;p-=p&-p) sum+=c[p];
```

于是我们就学会了 Fenwick Tree，即树状数组。是不是很好理解！

RMQ: Sparse Table

RMQ 问题是一类满足结合律和幂等律的区间查询问题的总称，比如：

- 区间最大/最小值
- 区间查询最大公因数
- 区间按位与/或

显然可以线段树 $O(n) \rightarrow O(\log n)$ 解决。我们考虑这样一件事：对于区间 $[l, r]$ ，令 $d = \lfloor r - l + 1 \rfloor$ ，则这个区间一定能被分解为两个区间 $[l, l + 2^d - 1], [r - 2^d + 1, r]$ 的答案的合并。证明是 trivial 的，不再赘述。因此我们预处理所有长度为 2^k 的区间的答案（这一步是容易的），每次查询即可。复杂度：预处理 $O(n \log n)$ ，查询 $O(1)$ 。

```

for (int i = 1; i <= n; i++) cin >> f[0][i];
for (int j = 1; j <= logN; j++)
    for (int i = 1; i + (1 << j) - 1 <= n; i++)
        f[j][i] = max(f[j - 1][i], f[j - 1][i + (1 << (j - 1))]);
int s = Logn[y - x + 1];
cout << max(f[s][x], f[s][y - (1 << s) + 1]) << '\n';

```

能否线性预处理？

能！

我们先考虑这样一个问题：如果序列满足这样一条特殊性质：

- $a_i - a_{i-1} \in \{1, -1\}$

我们同样采用分块的思想，将原序列分为长度为 B 的块。我们考虑这样一件事：

- 对于长度和差分序列也相同的块，最值所处的相对位置一定相同，这个时候我们认为两个块**本质相同**；
- 对于本质相同的块，它的所有开头结尾的相对位置相同的子段，也一定是**本质相同**的。

举个例子，**1 2 3** | **4 5 6** 就是两个本质相同的块，可以发现里面的最小值都是块内的第一个数，最大值是块内的第三个数。

我们注意到：

- 本质不同的差分序列有 2^B 种；
- 因此本质不同的块子段最多不可能超过 $2^B \cdot B^2$ 种。

对于整块，可以使用 ST 表进行 $O(\frac{n}{B} \log \frac{n}{B})$ 和 $O(1)$ 的查询；对于散块部分也可以进行 $O(2^B B^2)$ 的预处理和 $O(1)$ 的查询。

因此，预处理的复杂度为 $O(\frac{n}{B} \log \frac{n}{B} + 2^B B^2)$ 。我们取 $B = 0.5 \log n$ ，可以得到 $O(\frac{n}{\log n} \log \frac{n}{\log n} + \sqrt{n} \cdot \log^2 n) = O(n)$ 的复杂度。通过预处理时记录答案的相对位置，我们不难同时实现对答案所处的位置（序列坐标）的查询。

最近公共祖先的更优解法

那这个带有限制的 rmq 问题对于解决没有限制的问题有什么作用呢？我们再考虑如下问题：

- 查询树上两点的公共祖先。

为了解决这个问题，我们需要借助一项工具：**欧拉序**。我们递归定义树的欧拉序如下：

- 空树的欧拉序 $E(\emptyset)$ 为空；
- 对于大小为 1，根为 u 的子树，它的欧拉序为 u ；
- 假设某棵子树的根节点编号为 u ，且儿子序列为 $v_1, v_2, \dots, v_m$ ，则欧拉序 $E(\text{Subtree}_u)$ 为

$$u + E(\text{Subtree}_{v_1}) + u + E(\text{Subtree}_{v_2}) + u + \dots + u + E(\text{Subtree}_{v_m}) + u$$

我们记录欧拉序所对应的节点深度序列，并使用上述算法进行维护，即可查询欧拉序的某一段上深度最小的点。设点 x 第一次出现的位置为 $\text{fir}(x) = \min\{p : E(T)_p = x\}$ ，我们证明如下 Lemma：

- 对于点 u, v ，欧拉序上 $\text{fir}(u)$ 和 $\text{fir}(v)$ 之间深度最小的点**唯一**，且为它们的 LCA。

不妨设 LCA 为 w ，我们先证明**存在性**：

- 如果 $w = u$ ，那么 $E_{\text{fir}(u)} = u = w$ 显然成立。
- 如果 $w \neq u$ ，那么 u 和 v 位于 w 的不同儿子子树中。DFS 从第一次访问 u 到第一次访问 v ，必须先从 u 所在子树回溯到 w ，然后再进入 v 所在子树。按照欧拉序定义，每次从儿子返回父亲时都会记录父亲，所以中间一定会记录到 w 。

因此 w 至少出现一次。再证区间中不会出现比 w 更浅的结点，显然根据 dfs 的性质，在 dfs 完子树之前不会回到祖先；而要想前往一个不是祖先的更浅节点，这个节点必然不存在于 w 的子树内，也就是必然会离开 w 的子树，矛盾。

最后证深度等于 $\text{dep}(w)$ 的结点只能是 w 。

因为在从 $\text{fir}(u)$ 到 $\text{fir}(v)$ 的过程中，DFS 始终没有离开 w 的子树。于是区间中的所有结点都在 w 的子树内。

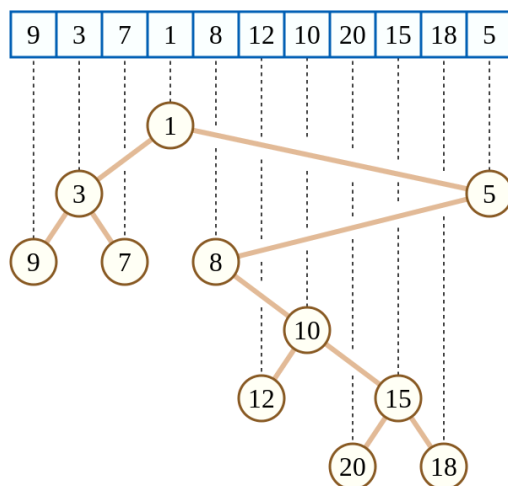
在 w 的子树中，深度等于 $\text{dep}(w)$ 的结点只有 w 自己；其他真后代的深度都严格大于 $\text{dep}(w)$ 。

因此若 $\text{dep}(E_t) = \text{dep}(w)$ 必有 $E_t = w$ 。

于是我们就有了一个 $O(n) - O(1)$ 的 LCA 算法！

回到 RMQ 问题：笛卡尔树

为了避免出现歧义，我们将 rmq 问题的元素换成有序二元组 $q_i = (a_i, i)$ ，其中 $q_i < q_j$ 当且仅当 $a_i < a_j$ ，或者 $a_i = a_j$ 时有 $i < j$ 。



小根笛卡尔树递归定义如下：

对任意区间 $[l, r]$ ，如果 $l > r$ ，对应空树；否则令 m 是 $[l, r]$ 中使 (a_m, m) 最小的位置。然后：

- m 是这棵子树的根；
- 左子树递归地由区间 $[l, m - 1]$ 构成；
- 右子树递归地由区间 $[m + 1, r]$ 构成。

整棵笛卡尔树就是区间 $[1, n]$ 对应的树。我们接下来证明笛卡尔树的中序遍历正好是 $1, 2, \dots, n$ 。

对区间长度做归纳。如果区间 $[l, r]$ 为空或只有一个点，结论显然成立。假设任意长度小于 $r - l + 1$ 的区间都成立。现在考虑区间 $[l, r]$ 。根据递归定义，根是 m ，左子树来自 $[l, m - 1]$ ，右子树来自 $[m + 1, r]$ 。由

归纳假设，左子树中序遍历为 $l, l+1, \dots, m-1$ ，右子树中序遍历为 $m+1, m+2, \dots, r$ 。因此整棵树的中序遍历是 $l, l+1, \dots, r$ 。

所以对整棵树 $[1, n]$ ，中序遍历就是 $1, 2, \dots, n$ 。

我们还能发现任意父亲 x 和儿子 y ，都有 $(a_x, x) < (a_y, y)$ 。这一点是显然的。

现在给定一次 RMQ 查询 $[l, r]$ ，要求找 $a_l, \dots, a_r$ 中最小值的位置。

令 $z = \text{LCA}(l, r)$ ，我们证明 z 就是 $[l, r]$ 的最小值位置。因为 z 是 l 和 r 的最近公共祖先，所以 l 和 r 都在 z 的子树中。由上面证明过的性质， z 的子树对应数组中的某个连续区间 $[L_z, R_z]$ 。

既然 l, r 都在这棵子树里，并且子树对应连续区间，那么整个 $[l, r]$ 都包含在 $[L_z, R_z]$ 中。

又因为 z 是这棵子树的根，而递归定义保证每棵子树的根都是对应区间的最小值位置，所以 z 不大于 $[L_z, R_z]$ 中的所有点，当然也不大于 $[l, r]$ 中的所有点。

还需要说明 z 本身确实在 $[l, r]$ 中。

这由中序遍历性质得到。因为中序遍历顺序就是数组下标顺序，所以：

- 如果 $z = l$ 或 $z = r$ ，显然 $z \in [l, r]$ ；
- 否则， l 和 r 分别落在 z 的左右子树中，于是必有 $l < z < r$ 。

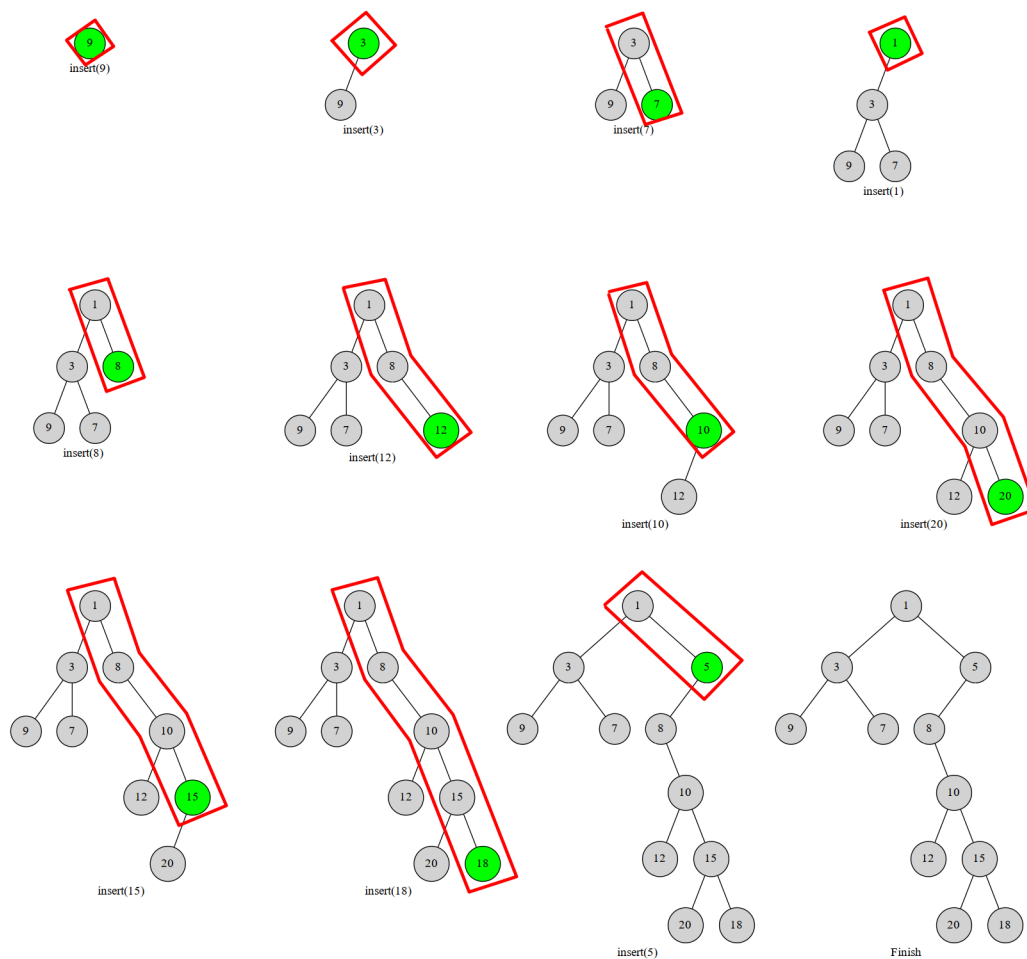
所以总有 $z \in [l, r]$ 。

于是 z 既是 $[l, r]$ 中的一个位置，又不大于 $[l, r]$ 中的所有位置，因此它就是 $[l, r]$ 的最小值位置。所以有 $\text{RMQ}(l, r) = \text{LCA}(l, r)$ 。

那么怎么构造笛卡尔树呢？好问题！我们考虑将元素按 k 升序依次插入到当前的笛卡尔树中，对于一棵笛卡尔树，定义「右链」为从根节点开始一直走右儿子，走到叶节点形成的链。则插入节点后，这个节点一定在右链上。因为是按照满足 BST 性质的 k 升序插入，那么这个新插入的节点必然在树的**最右端**。这个节点不可能是一个左儿子，也没有右儿子。

于是我们执行这样一个过程，从下到上比较右链节点和当前节点 u 的 w ，如果找到了一个右链上的节点 x 满足 $w_x < w_u$ 就把 u 设为 x 的右儿子，而原本的右子树成为 x 的左子树。

显然每个数最多进出右链一次（或者说每个点在右链中存在的是一段连续的时间）。这个过程可以用单调栈维护，栈中维护当前笛卡尔树的右链上的节点。一个点不在右链上了就把它弹掉。这样每个点最多进出一次，复杂度 $O(n)$ 。



于是通过上述的步骤，我们就得到了一个 $O(n) - O(1)$ 的 rmq 算法！